

PROYECTO DE CURSO

Documento de Apoyo 03: (Semana 8)

750021C - DESARROLLO DE SOFTWARE II



FACULTAD DE INGENIERÍA
Escuela de Ingeniería de Sistemas y
Computación

FEBRERO - JUNIO / 2026

Construcción Automática del Pipeline

1. Objetivos

- 1.1. Implementar el “Pipeline como Código” mediante la creación de un archivo **Jenkinsfile**.
- 1.2. Configurar el primer Job automatizado en el VPS que responda a eventos del repositorio.
- 1.3. Automatizar el proceso de construcción (Build) de imágenes Docker desde Jenkins.

2. Información Clave

Es necesario entender que el Jenkinsfile es el manual de instrucciones del servidor. Para ello utilizaremos:

- **Declarative Pipeline:** Sintaxis de Jenkins basada en bloques (pipeline, agent, stages).
- **Agent Any:** Indica que Jenkins puede usar cualquier ejecutor disponible en el VPS.

- **Source Control Management (SCM):** Configuración que permite a Jenkins descargar el código desde GitHub automáticamente.
- **Docker Socket:** Permiso especial que ya tiene el Jenkins del VPS para ejecutar comandos **docker** internos.

3. Procedimientos

A. Creación del Jenkinsfile

En la raíz del proyecto debe existir un archivo llamado exactamente **Jenkinsfile**, sin extensión y con la primera letra mayúscula. Ejemplo base:

Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "uniticket-grupo-X"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build Image') {
            steps {
                script {
                    sh "docker build -t ${IMAGE_NAME}:latest ."
                }
            }
        }
    }
}
```

B. Configuración del Job en el VPS

1. Ingresar a su instancia de Jenkins `http://45.55.145.98:8080`
2. Crear un **Nuevo Elemento** de tipo **Pipeline**.
3. En la sección **Build Triggers**, marcar: **GitHub hook trigger for GITScm polling**.
4. En la sección **Pipeline**, seleccionar: **Pipeline script from SCM**.
 - **SCM:** Git.
 - **Repository URL:** La URL del repositorio en GitHub.
 - **Credentials:** Seleccionar la credencial (PAT) creada con el documento de la semana 4.
 - **Branch Specifier:** `*/develop`.

C. Activación del Disparador

Como plan inicial antes de los Webhooks completos, es necesario configurar el Polling SCM.

1. En la configuración del Job, ir a **Consultar repositorio (Polling SCM)**.
2. Programación: `H/2 * * * *`
(Esto le dice a Jenkins que revise cambios cada 2 minutos).

4. Resultados Esperados

- Al hacer un `git push` a la rama `develop`, Jenkins debe detectar el cambio automáticamente en máximo 2 minutos.
- La consola de Jenkins debe mostrar el historial de ejecución en color verde indicando el éxito (Success).
- Se debe generar una nueva imagen Docker en el VPS con el nombre del grupo.

5. Solución de Problemas

Problema	Posible Causa	Solución Sugerida
Error: Permission denied (docker.sock)	El contenedor de Jenkins no tiene permisos de root para usar Docker.	Notificar al docente para revisar los permisos del volumen <code>/var/run/docker.sock</code> .
Jenkins no detecta el push	La rama configurada en el Job no coincide con la rama donde se subió el código.	Verificar que el Job apunte a <code>develop</code> y no a <code>main</code> .
Error de sintaxis en el Jenkinsfile	Olvido de una llave <code>{ }</code> o comillas <code>" "</code> mal cerradas en el código Groovy.	Revisar los logs de "Console Output" en Jenkins; ahí indica la línea exacta del error.